

Vision Transformers

Theoretical Overview and Fine-tuning to CIFAR-10 dataset

Thisanda Ayuka Thelikada Palliya Guruge

July 30, 2026

1. Introduction

Transformer architecture is a very popular and highly scalable deep learning architecture used in modern chatbots such as ChatGPT. Originally designed for Natural Language processing tasks, a paper by Dosovitskiy et al. (2020), explored the use of transformers in vision tasks. Results from this work illustrate that transformers used in computer vision so-called Vision transformers (ViT) are able to complete vision tasks with high accuracy compared to previously used Convolutional Neural Networks. Since this discovery a large amount of scientific research has been conducted on understanding ViTs its limitations, advantages and possible real-world applications of ViTs (Aburass et al., 2025; Ghosh & Goswami, 2026; Zhang et al., 2026). One of the successful uses of ViTs has been in the field of medical imaging, specifically in breast cancer and COVID-19 (Ayana et al., 2023; Mehboob et al., 2022). ViTs have also been used in autonomous driving, for the identification of obstacles, show the promise of its future applications (Sun et al., 2022).

Therefore, this paper aims to investigate the capability of vision transformers by fine-tuning it for a small dataset. We compare how the accuracy of a fine-tuned model evolves with the number of samples provided and the number of training epochs.

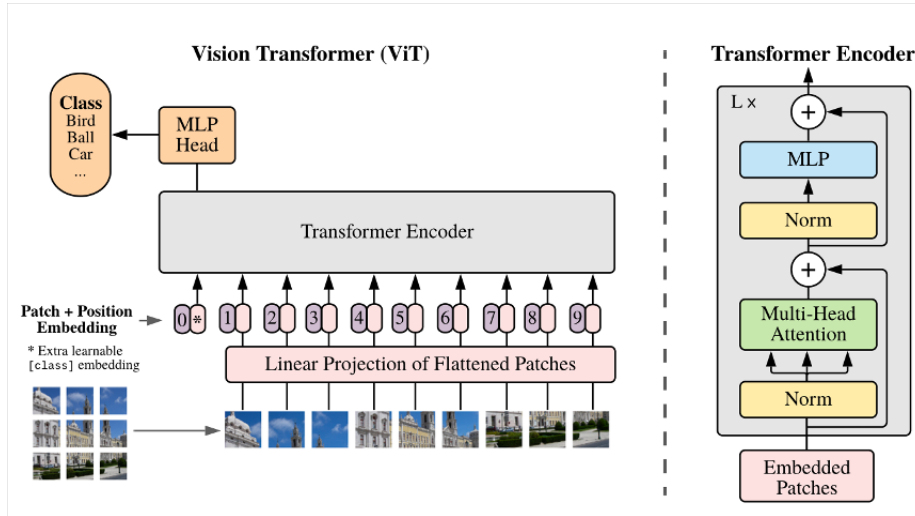


Figure 1.1: Structure of the transformer encoder and ViT taken from Dosovitskiy et al. (2020).

2. Theory

In this section, the processes within a ViT are discussed. The section explores how images are converted to 1D embedded vectors to be input into transformer encoder and then classified to their respective classes.

2.1 Patch Embedding

The image is split into N number of patches. Each patch has C channels therefore in order to convert into 1D embedded vectors, each channel is flattened. This is done by concatenating a row of pixels to the row underneath. Finally, all 1D vectors of all channels are concatenated to form a 1D representation of a patch x_p^n where $n \in 1, \dots, N$. The image, x can be considered as, $x \in \mathbb{R}^{H \times W \times C}$ has to be embedded into a vector of model dimension D where it is handed to the transformer encoder. For this each 1D representation of a patch is linearly projected using a learnable parameter $\mathbf{E}$. As for the first row of the input tensor a learnable class vector $x_{[\text{class}]}$ is concatenated to the input tensor. This vector is used similarly to an NLP transformer, but in this case, responsible for collecting information about other patches and is used to classify the image in the classification stage. To the class token and linearly projected embedded vectors, positional information is added via positional encoding x_{pos} .

$$z_0 = [x_{[\text{class}]}; x_p^1 \mathbf{E}; \dots; x_p^N \mathbf{E}] + x_{\text{pos}} \quad x_p \in \mathbb{R}^{(P^2 \cdot C) \cdot D} \text{ and } x_{\text{pos}} \in \mathbb{R}^{(N+1) \times D} \quad (2.1)$$

2.2 Transformer Encoder

There are L number of encoder layers, the following equations illustrate the processes taking place within the transformer which include Layer Normalisation (LN), Multi-head Self Attention (MSA), and Multi-Layer Perceptron (MLP) blocks.

$$z'_l = \text{MSA}(\text{LN}(z_{l-1})) + z_{l-1} \quad l = 1, \dots, L \quad (2.2)$$

$$z_l = \text{MLP}(\text{LN}(z'_l)) + z'_l \quad l = 1, \dots, L \quad (2.3)$$

2.2.1 Multi-Head Self Attention

Self-Attention is the process where contextual relations between different tokens and embedded vectors are looked upon of the input (Lai et al., 2020). The ViT builds these contextual relationships between the embedded vectors and quantify these. There are typically multiple heads of SA, but for simplicity we focus on single head for which there are three learnable tensors defined W_Q , W_K and W_V . The product of these learnable tensors with the input tensor to give query(q), key(k) and value(v) shown in equation 2.4. Self Attention is calculated as shown in equation 2.5. D_h is the dimension of an attention head. For a single head attention $D_h = D$.

$$q = zW_Q \quad k = zW_K \quad V = zW_V \quad (2.4)$$

$$\text{SA}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{D_h}}\right) V \quad (2.5)$$

Multi-head self attention applies single attention a k number of times and then concatenates the results together as shown in equation 2.6. A single head dimension D_h is chosen such that $D_h = \frac{D}{k}$.

$$\text{MSA}(a) = \text{Concat}(\text{SA}_1(a), \dots, \text{SA}_k(a)) \quad \text{MSA}(a) \in \mathbb{R}^{(N+1) \times D} \quad (2.6)$$

Where a is an arbitrary input.

2.2.2 Multi-layer Perceptron

A multi-layer perceptron takes the values from the MSA block, after normalisation, and interprets these values. MLP in each encoder layer has one hidden layer with GeLU non-linearity as the activation function. During the training process, the weights of each layer is changed.

2.3 Classification Multi-layer Perceptron and [class] Token

After the L^{th} layer, the output of the final layer is considered to predict the class that the image belongs to. To do this, only the class token, z_L^0 , of the layer normalised output matrix is considered. y determines the predicted class by the position of its largest entry shown in equation in 2.7.

$$y = \text{MLP}(\text{LN}(z_L^0)) \quad (2.7)$$

During the pre-training stage, the final head of MLP is a network consisting of a single hidden layer, one input and an output layers. The activation function used here is the tanh non-linearity. Output layer has a number of neurons equal to the number of classes in the training dataset. The MLP is trained to minimise the cross-entropy loss, and the gradient of the loss function is back-propogated into the layers below, that is used to optimise all learnable parameters and weights.

During the fine-tuning stage the process is changed, a simpler MLP is used as the transformer's final head, one that is linear (consisting of merely an input and output layer). We again use tanh non-linearity. Much like the pre-training classifier MLP, the network is trained to minimise the cross-entropy loss, however, all encoder layers are frozen and therefore gradient descent does not further optimise the parameters of the encoder layers.

3. Methodology

To investigate the capability of ViTs, a ViT is fine-tuned using the CIFAR-10 dataset. The aim is to investigate how the model accuracy changed with the change in the number of training epochs and the number of samples used during training. Below, the specific model chosen and dataset are discussed along with the algorithm used for the fine-tuning process.

3.1 Pre-trained Model and Fine-tuning Dataset

The model used is based on the ViT architecture of Dosovitskiy et al. (2020), developed by OpenAI Radford et al. (2021) and pre-trained on the public image dataset YFCC.100M by Thomee et al. (2016) from YahooLabs. Important model properties are shown in table 3.1.

CIFAR-10 dataset is chosen for the fine-tuning process, and it is an openly available image dataset that contains 600 32×32 images from 10 different classes each providing a total of 60 000 images. The dataset contains 50 000 training images(considered samples) and 10 000 testing images (Krizhevsky, Hinton, et al., 2009).

Table 3.1: Model information for the ViT/B 32 model used.

Model Variant	Number of layers	Hidden Size D	Input Patch size
ViT/B 32	12	768	32×32

3.2 Algorithm Design

The program used for the fine-tuning process is written in Python. Python libraries numpy, torch, transformers, HuggingFace and scikit-learn were used. HuggingFace contains an amalgamation of pre-trained models.

The algorithm(See appendix B for the Python code) begins by importing the necessary libraries that were listed. A function was written to train and test the ViT, while another function was implemented to call back the trainer to report the accuracy scores at each epoch during training and testing. Additional custom functions such as collation functions were developed for the trainer, but this will not be discussed in detail further. Within the function defined to run and test the ViT, the CIFAR-10 dataset is imported via HuggingFace and then data is split into training and testing data. The training data is then further split and only the required number of samples are selected. However, the testing dataset remains the same. The intention is to vary the number of training samples but test it on all 10 000 testing images. The model was imported from CLIP and then both training and testing data was transformed to the format needed by the model. A new classifier was initialised as explained earlier. A variable was used to store all trainer information which also includes the number of training epochs. This information was provided and the model was run. Following the training process, the function would now test the ViT using the testing data and produce files containing the testing accuracies at different number of epochs.

In the main program, a loop was created that iterates from 10 to 50 000 sample and fine-tunes the model again for each different sample size. Therefore, in total several files are produced that contain the testing accuracies with 20 epochs for a number of different sample sizes.

4. Results and Discussion

Firstly, it can be observed from figure 4.1 that generally as the number of epochs increase the accuracy of the model has increased. As the number of samples used in training increases, model reaches higher accuracy with a lower number of epochs. There are a few fluctuations visible, where this is not seen. This could be prevented by repeating the process multiple times along with the use of error bars, which would mitigate such fluctuations and give a much smoother curve. However, this could not be done due to the lack of time and the associated computational resources needed.

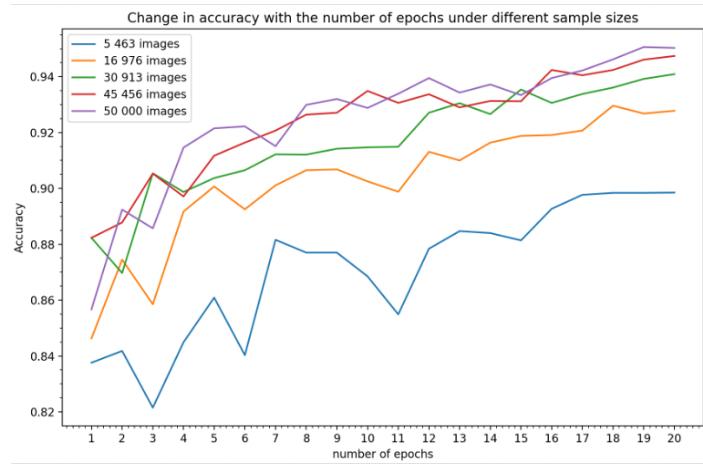


Figure 4.1: Test Accuracy of the fine-tuned model as a function of the training epochs and different sample sizes.

On the other hand as shown in figure 4.2, as the number of samples are increased, until ≈ 10000 samples, the accuracy increases significantly in an exponential plateau fashion. Beyond 10 000 samples, rate of increase in test accuracy decreases and linear with number of samples used.

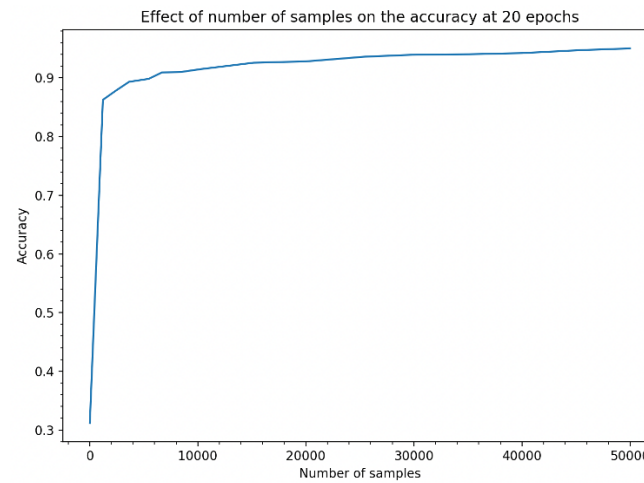


Figure 4.2: Model test accuracy on CIFAR-10 as a function of the number of samples used in training at 20 epochs.

5. Conclusion

As discussed in the previous sections, ViTs use a standard NLP encoder-only transformer for computer vision tasks, in this case specifically image classification. Pre-trained models such as the one used for this piece of work show that they are capable of achieving high accuracy without requiring a lot of data in the fine-tuning stage. The self attention mechanism in ViTs is global allowing to capture long range relationships between patches which makes this architecture powerful.

This also highlights a key number of improvements that could be made for any future work. Firstly, it was noted that the ViT model used, creates patches of size 32×32 which is also the size of each CIFAR-10 image. Future work could be done in fine-tuning different variants of the model created by Dosovitskiy et al. (2020).

For a more qualitative comparison of the ViT with CNNs, the same set of tests could be carried out by a ViT and CNN with similar pre-training and using CIFAR-10 or any standard image dataset. This would offer the opportunity to compare whether the effects of size of the fine-tuning dataset to the performance of the two models. It is also important that ViTs and CNNs of similar complexity is used for this. For example, the base model of the ViT would have to be compared to the base CNN (i.e. ResNet).

Furthermore, different training methods could also be explored such as self-supervised learning capabilities of the ViT. Within this scope the training of the model with captions is also something that could be explored.

In this piece of work, we use a pre-trained ViT model by Radford et al. (2021) and fine-tune the model for CIFAR-10 dataset. To summarise observed results, we see that the test accuracy of the model increases as the number of training epochs and number of samples used in training increase. Notably, it is observed that beyond a certain number of samples, the increase in test accuracy was minimal.

Bibliography

- Aburass, S., Dorgham, O., Al Shaqsi, J., Abu Rumman, M., & Al-Kadi, O. (2025). Vision transformers in medical imaging: A comprehensive review of advancements and applications across multiple diseases. *Journal of Imaging Informatics in Medicine*, 38(6), 3928–3971.
- Ayana, G., Dese, K., Dereje, Y., Kebede, Y., Barki, H., Amdissa, D., Husen, N., Mulugeta, F., Habtamu, B., & Choe, S.-W. (2023). Vision-transformer-based transfer learning for mammogram classification. *Diagnostics*, 13(2). <https://doi.org/10.3390/diagnostics13020178>
- Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer normalization. <https://arxiv.org/abs/1607.06450>
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*.
- Ghosh, D., & Goswami, A. (2026). Semantic-augmented vision transformer with cross-modal fusion for waste classification. *Applied Soft Computing*, 190, 114602. <https://doi.org/10.1016/j.asoc.2026.114602>
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep residual learning for image recognition. <https://arxiv.org/abs/1512.03385>
- Krizhevsky, A., Hinton, G., et al. (2009). Learning multiple layers of features from tiny images.
- Lai, Q., Khan, S., Nie, Y., Sun, H., Shen, J., & Shao, L. (2020). Understanding more about human and machine attention in deep neural networks. *IEEE Transactions on Multimedia*, 23, 2086–2099.
- Mehboob, F., Rauf, A., Jiang, R., Saudagar, A. K. J., Malik, K. M., Khan, M. B., Hasnat, M. H. A., AlTameem, A., & AlKhathami, M. (2022). Towards robust diagnosis of covid-19 using vision self-attention transformer. *Scientific Reports*, 12(1), 8922.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., . . . Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. <https://arxiv.org/abs/1912.01703>
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. <https://arxiv.org/abs/2103.00020>
- Sun, Z., Liu, C., Qu, H., & Xie, G. (2022). Pvformer: Pedestrian and vehicle detection algorithm based on swin transformer in rainy scenes. *Sensors*, 22(15). <https://doi.org/10.3390/s22155667>
- Thomee, B., Shamma, D. A., Friedland, G., Elizalde, B., Ni, K., Poland, D., Borth, D., & Li, L.-J. (2016). Yfcc100m: The new data in multimedia research. *Commun. ACM*, 59(2), 64–73. <https://doi.org/10.1145/2812802>
- Zhang, T., Li, L., Zhou, Y., Liu, W., Qian, C., Hwang, J.-N., & Ji, X. (2026). Cas-vit: Convolutional additive self-attention vision transformers for efficient mobile applications. *IEEE Transactions on Image Processing*, 35, 1899–1909. <https://doi.org/10.1109/TIP.2026.3655121>

A. Appendix - Python script

```
1 #Importing necessary libraries
2 import torch
3 import torchvision
4 import matplotlib.pyplot as plt
5 from transformers import TrainingArguments, Trainer,
   AutoModelForImageClassification, TrainerCallback
6 from transformers import CLIPProcessor, CLIPModel
7 from datasets import load_dataset
8 import numpy as np
9 from sklearn.metrics import accuracy_score
10 from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
11
12
13 #function that takes in our cifar-10 dataset and creates a dictionary consisting
   of pixel values
14 #followed by the label of the image.
15 def collate_fn(examples):
16     pixels = torch.stack([example["pixel_values"] for example in examples])
17     labels = torch.tensor([example["label"] for example in examples])
18
19     return {"pixel_values": pixels, "labels": labels}
20
21
22 #Function that takes logits, identifies the one with the highest probability and
   creates an accuracy score
23 #in comparison to the ground truth.
24 def compute_metrics(eval_pred):
25     if hasattr(eval_pred, "predictions"):
26         preds = eval_pred.predictions
27         labels = eval_pred.label_ids
28     else:
29         preds, labels = eval_pred
30
31     preds = np.argmax(preds, axis=1)
32     return {"accuracy": accuracy_score(labels, preds)}
33
34 class EpochAccCallback(TrainerCallback):
35     def __init__(self, train_dataset, test_dataset):
36         self.train_dataset = train_dataset
37         self.test_dataset = test_dataset
38         self.train_curve = [] # list of (epoch, acc)
39         self.test_curve = []
40         self.trainer = None # will be set after Trainer is created
41
42
43     def on_epoch_end(self, args, state, control, **kwargs):
44         if self.trainer is None:
45             return control
46
47         epoch = int(round(state.epoch)) if state.epoch is not None else None
48
```

```

49     old_eval_bs = self.trainer.args.per_device_eval_batch_size
50     try:
51         # temporarily increase eval/predict batch size
52         self.trainer.args.per_device_eval_batch_size = 5000
53
54         # force dataloader rebuilds with the new batch size if cached
55         if hasattr(self.trainer, "_eval_dataloader"):
56             self.trainer._eval_dataloader = None
57         if hasattr(self.trainer, "_test_dataloader"):
58             self.trainer._test_dataloader = None
59
60         train_metrics = self.trainer.evaluate(
61             eval_dataset=self.train_dataset,
62             metric_key_prefix="train",
63         )
64
65         test_metrics = self.trainer.predict(
66             test_dataset=self.test_dataset,
67             metric_key_prefix="test",
68         ).metrics
69
70     finally:
71         # restore original eval batch size
72         self.trainer.args.per_device_eval_batch_size = old_eval_bs
73
74         # clear again so future eval/predict uses the restored batch size
75         if hasattr(self.trainer, "_eval_dataloader"):
76             self.trainer._eval_dataloader = None
77         if hasattr(self.trainer, "_test_dataloader"):
78             self.trainer._test_dataloader = None
79
80     self.train_curve.append((epoch, train_metrics.get("train_accuracy")))
81     self.test_curve.append((epoch, test_metrics.get("test_accuracy")))
82     return control
83
84 #Function to create files that contain epoch and the accuracy reached.
85 def write_curve_txt(path, curve):
86     with open(path, "w") as f:
87         f.write("epoch\taccuracy\n")
88         for ep, acc in curve:
89             if acc is None:
90                 f.write(f"{ep}\t\n")
91             else:
92                 f.write(f"{ep}\t{acc:.6f}\n")
93
94
95 #We create a function that runs the ViT model for cifar-10. We input into this
96 #function the number of training epochs
97 #For function generalisation, we will also have a variable that now changes the
98 #amount of training data vs testing data(Number of samples).
99 def ViT_fine_tuned(number_of_samples, no_of_training_epochs):
100
101     dataset = load_dataset("cifar10") #load_dataset('cifar10', split='train').
102     shuffle(seed=42)
103     testing_data = dataset["test"]
104     training_data_full = dataset["train"].shuffle(seed=42)
105     training_data = training_data_full.select(range(number_of_samples))
106
107     # Create a dictionary to track the images and the labels of all these images
108
109     # integer dictionary contains the label followed by what it is, can be used
110     # to convert between logits and human labels.

```

```

108     integer_dictionary = dict((k, v) for k, v in enumerate(training_data.
109 features['label'].names))
110     # string dictionary does the opposite order to be fed into the training
111     phase.
112     string_dictionary = dict((v, k) for k, v in enumerate(training_data.features
113 ['label'].names))
114
115     # Defining our pre-trained model
116     model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
117     processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32",
118 use_fast='True')
119
120     def batch_transform(batch):
121         # img.converts the image to RGB values
122         images = [img.convert('RGB') for img in batch['img']]
123
124         # this function utilises two other internal functions to do the
125         following
126         # 1. Take the images and resizes and re-normalises them. (Also returns a
127         tensor of pixel values)
128         inputs = processor(images=images, return_tensors='pt')
129
130         batch['pixel_values'] = inputs['pixel_values']
131
132         return batch
133
134     # The function batch_transform takes our dataset and changes into a
135     structure accepted into our model.
136     training_data.set_transform(batch_transform)
137     # valds.set_transform(batch_transform)
138     testing_data.set_transform(batch_transform)
139
140     # Rather than typing new classifier, I have used one that is created for
141     fine-tuning via some research
142     # also updated the number of labels as cifar-10 has only 10 classes and also
143     input the id2labels and labels2id
144     model = AutoModelForImageClassification.from_pretrained(
145         "openai/clip-vit-base-patch32",
146         num_labels=10,
147         ignore_mismatched_sizes=True,
148         id2label=integer_dictionary,
149         label2id=string_dictionary
150     )
151
152     training_arguments = TrainingArguments(
153         f"run_nsamples_{number_of_samples:06d}", # File name for the directory
154         to store training information
155         eval_strategy="no", # evaluate manually in the callback
156         save_strategy='no',
157         learning_rate=2E-5, # How fast the model learns
158         per_device_train_batch_size=10,
159         per_device_eval_batch_size=4,
160         num_train_epochs=no_of_training_epochs,
161         weight_decay=0.01, # Prevents overfitting
162         load_best_model_at_end=False,
163         logging_dir='logs',
164         remove_unused_columns=False,
165         report_to="none",
166         logging_strategy="no",
167     )
168
169     acc_cb = EpochAccCallback(training_data, testing_data)

```

```

162     trainer = Trainer(
163         model,
164         training_arguments,
165         train_dataset=training_data,
166         eval_dataset=None,
167         data_collator=collate_fn,
168         compute_metrics=compute_metrics,
169         tokenizer=processor,
170         callbacks=[acc_cb],
171     )
172
173
174
175     acc_cb.trainer = trainer
176
177     trainer.train()
178
179     return acc_cb.train_curve, acc_cb.test_curve
180
181
182 list_of_sample_sizes = np.linspace(10, 60000, 100, dtype=int)
183
184 for nsamples in list_of_sample_sizes:
185     train_curve, test_curve = ViT_fine_tuned(number_of_samples=int(nsamples),
186     no_of_training_epochs=20)
187
188     write_curve_txt(f"train_accuracy_nsamples{int(nsamples):06d}.txt",
189     train_curve)
190     write_curve_txt(f"test_accuracy_nsamples{int(nsamples):06d}.txt", test_curve
191 )
192
193 #Inspiration for the code was taken from supersjgk(2023) via a Medium post. Date
194   of access - 14/01/2026
195
196 #supersjgk. (2023, December 11). Fine-Tuning Vision Transformer with Hugging
197   Face and PyTorch. Medium. https://medium.com/@supersjgk/fine-tuning-vision-
198   transformer-with-hugging-face-and-pytorch-df19839d5396

```

Listing A.1: ViT accuracy script.

B. Appendix B - Additional materials

This Appendix covers additional diagrams which could not be put into the main text and are categorised by what process they describe.

B.1 Additional diagrams

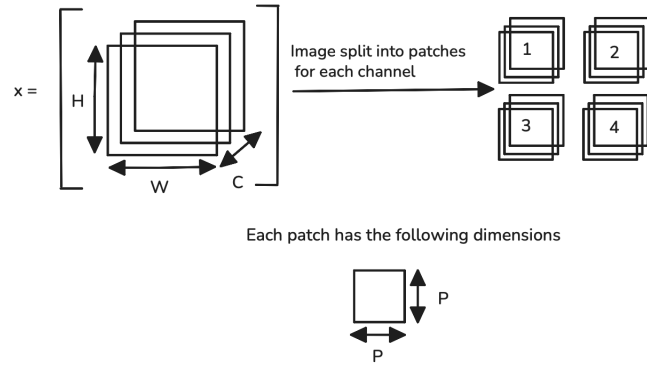


Figure B.1: Splitting image into patches

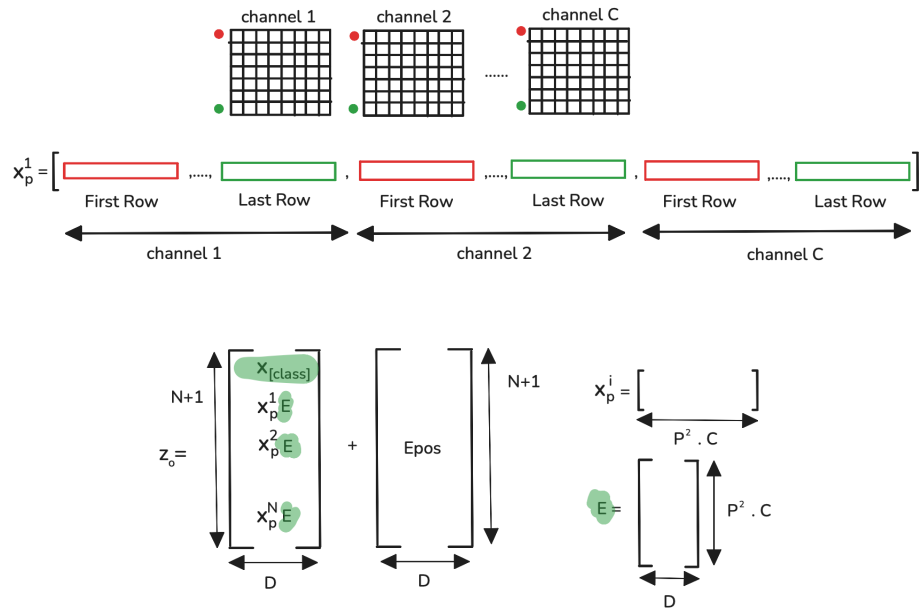


Figure B.2: Patch flattening

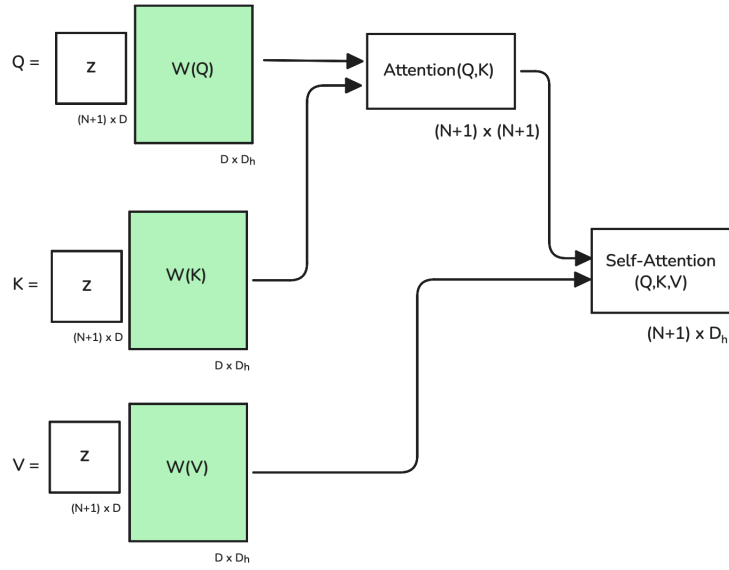


Figure B.3: Single Head attention

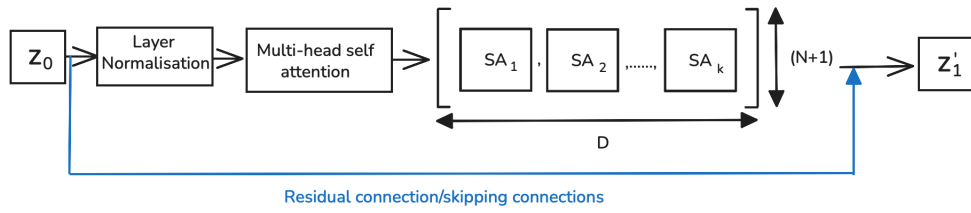


Figure B.4: Multi-Head Self Attention

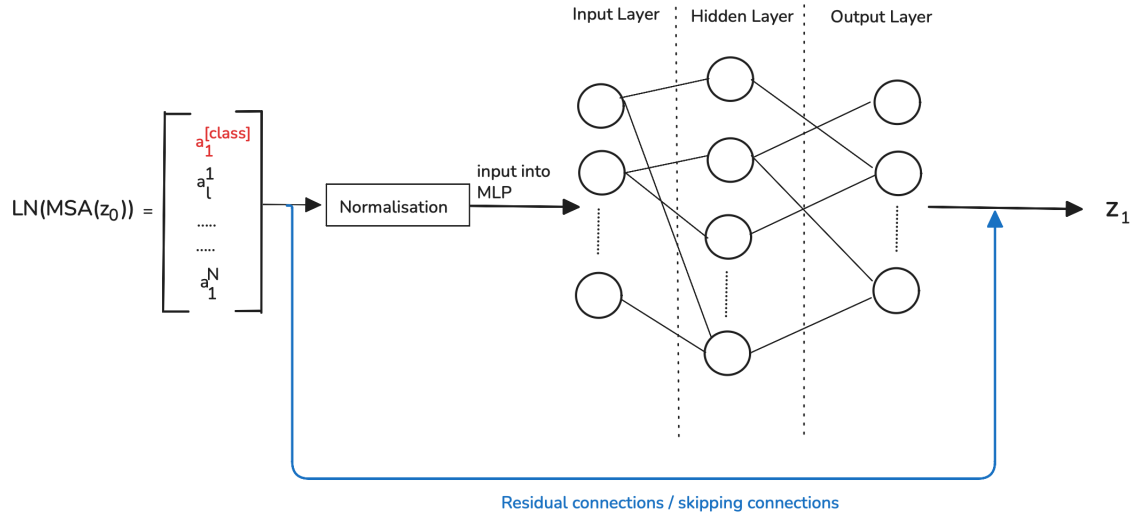


Figure B.5: MLP in the Encoder

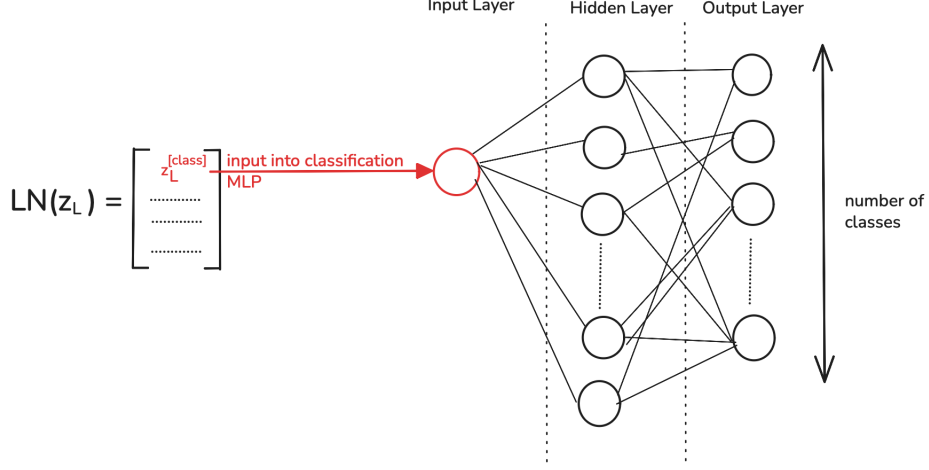


Figure B.6: Classification MLP in pre-training stage

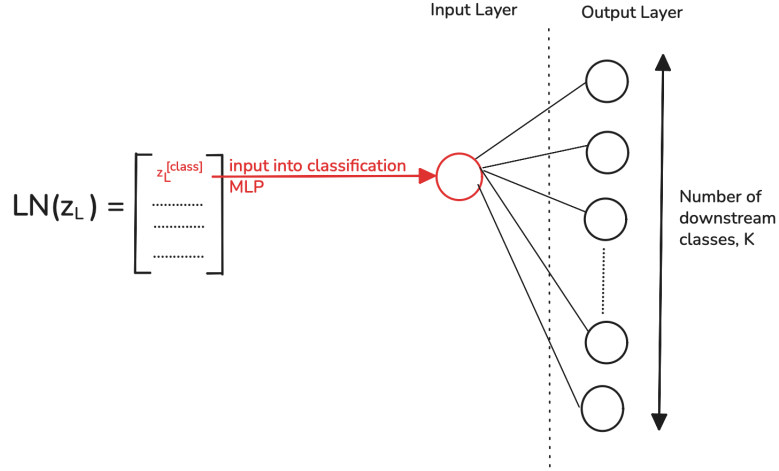


Figure B.7: Classification MLP in fine-tuning stage

B.2 Additional concepts

B.2.1 Layer Normalisation

Layer Normalisation(LN) takes a layer and subtracts each item by the mean and divides by the standard deviation of the layer (Ba et al., 2016). In this case, LN will take the mean and standard deviation of each column and every element will be transformed as shown in the equation below.

$$\text{LN}(z_{ij}) = \frac{z_{ij} - E[z]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma_j + \beta_j \quad (\text{B.1})$$

Where z_{ij} is the element-wise representation of z , $E[x]$ is the mean, $\text{Var}[x]$ is the variance in a column, $\epsilon = 1.0 \times 10^{-5}$ is value to prevent the fraction from being undefined, Note that learnable parameters γ and β are learned parameters of vector dimension D (Paszke et al., 2019).

B.2.2 Residual connections

Residual connections or skipping connections are important as these prevent the vanishing gradient problem He et al., 2015. Figure B.4 also shows the residual connections between the output of the MSA and the output of the MLP layer. Note that residual connections is the addition of an input layer to the output layer and shown as blue lines in the figures shown above.